

CS-334: Machine Learning

Lecture 04: Perceptron

09/08/2026

Prof. Shengpu Tang



Announcements

- **HW1** due Sun 11:59pm on Gradescope
 - Autograder: you can submit multiple times
- **In-Class Exercises** graded
 - Please check your score and submit regrade requests if you have question

ML Terminology: Loan Credit Risk Prediction

Features

How to make a **feature vector**

Feature space

Labels/Classes

ML Terminology: Loan Credit Risk Prediction

Features

- age: int between 0 and 120
- employment status: student, employed, unemployed

How to make a **feature vector**

- given an applicant, I'll ask their age and employment status, and the feature vector is [their age, their employment status]

Feature space

- the set of all possible combinations of age and employment status

Labels/Classes

- +1 for approve, -1 for not approve

ML Terminology: Loan Credit Risk Prediction

Features

- age $x_1 \in \{ \text{int between 0 and 120} \}$
- employment status $x_2 \in \{ \text{student, employed, unemployed} \}$

How to make a **feature vector**

- given an applicant i , I'll ask their age $x_1^{(i)}$ and employment status $x_2^{(i)}$,
and the feature vector is $\vec{x}^{(i)} = [x_1^{(i)}, x_2^{(i)}]$

Feature space

- $\{ \text{int between 0 and 120} \} \times \{ \text{student, employed, unemployed} \}$

Labels/Classes

- $y = +1$ for approve, $y = -1$ for not approve

Linear Classifier through the origin

$$h(\vec{x}; \vec{\theta}) = \text{sign}(\vec{\theta} \cdot \vec{x}) = \begin{cases} +1 & \text{if } \vec{\theta} \cdot \vec{x} > 0 \\ 0^* & \text{if } \vec{\theta} \cdot \vec{x} = 0 \\ -1 & \text{if } \vec{\theta} \cdot \vec{x} < 0 \end{cases}$$

feature vector

$$\vec{x} \in \mathbb{R}^d$$

$$\vec{\theta} \in \mathbb{R}^d$$

parameter vector
weight vector
normal vector
model coefficients

When making predictions,
since the output must be binary,
we can make 0 either +1 or -1.

Linear Decision Boundary

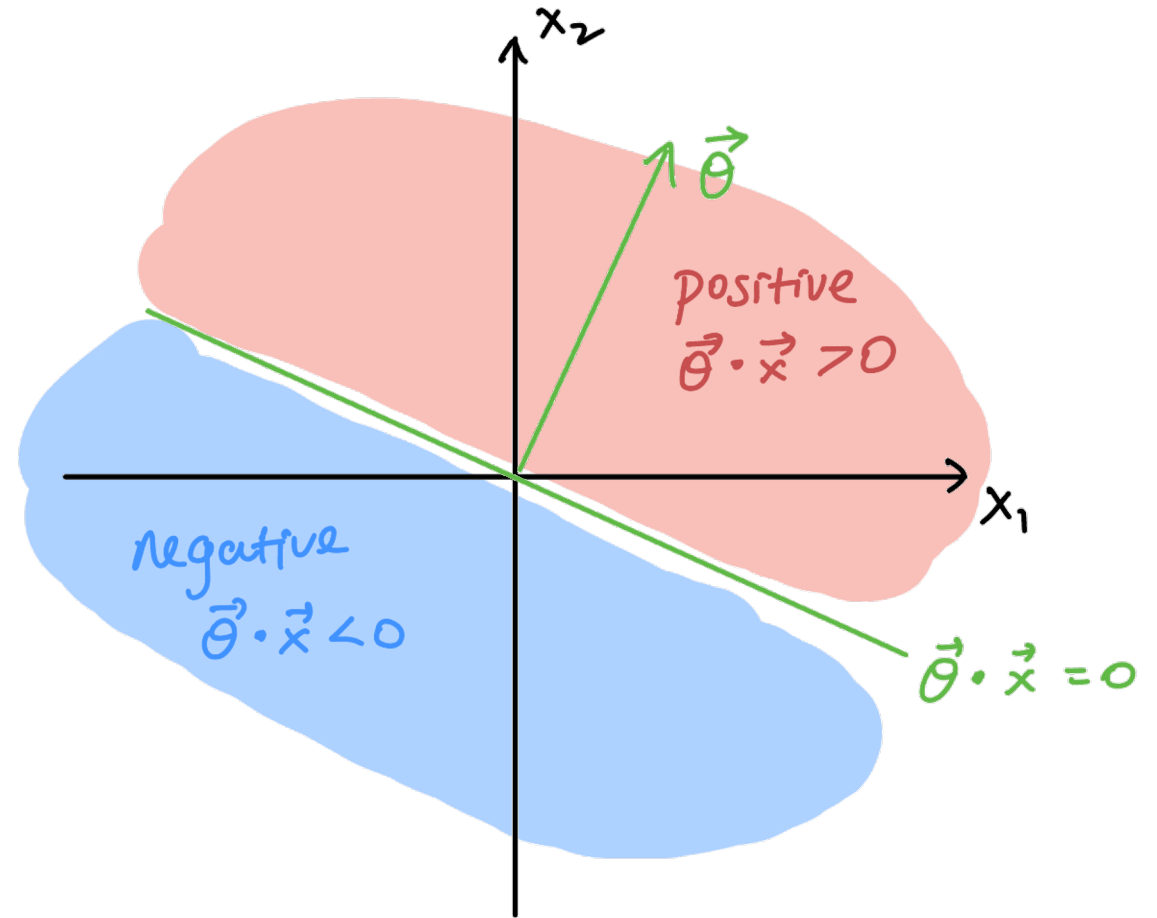
$$h(\vec{x}; \vec{\theta}) = \text{sign}(\vec{\theta} \cdot \vec{x})$$

It all comes down to the dot product

$$\vec{\theta} \cdot \vec{x}$$

$\vec{\theta}$ defines a hyperplane in d -dimensional space that goes through the origin

What does this look like in 3D?



Training Error

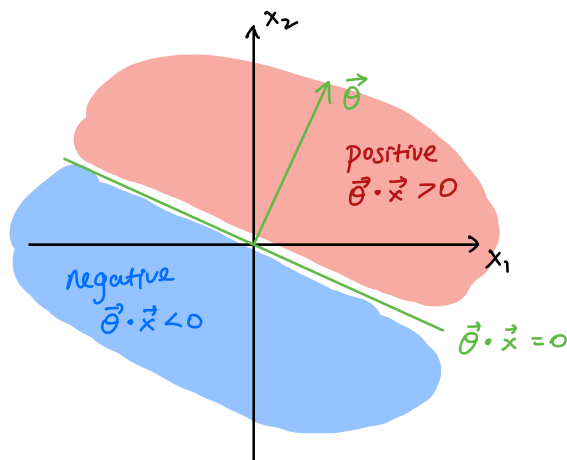
$$\mathcal{E}_N(\vec{\theta}) = \underbrace{\frac{1}{N}}_{\text{fraction of training examples}} \sum_{i=1}^N \underbrace{\mathbb{1}[y^{(i)} (\vec{\theta} \cdot \vec{x}^{(i)}) \leq 0]}_{\text{where the classifier predicts the wrong labels}}$$

Points on the decision boundary are considered misclassified

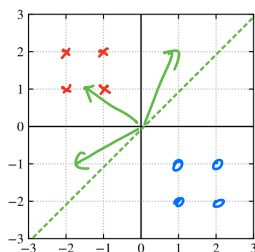
Lecture #4: Perceptron

Recap: linear classifiers through the origin.

$$h(\vec{x}; \vec{\theta}) = \text{sign}(\vec{\theta} \cdot \vec{x}) \quad , \quad \vec{x} \in \mathbb{R}^d, \quad \vec{\theta} \in \mathbb{R}^d.$$

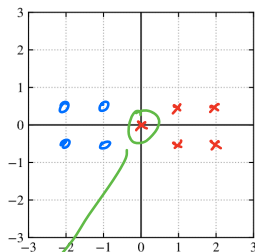


Examples from last lecture.



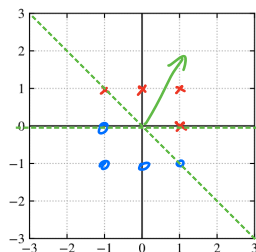
(a)

vector
of any length
pointing toward
upper left.



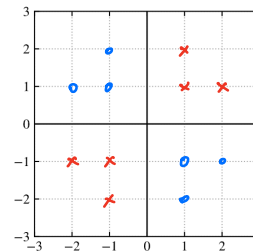
(b)

not possible
because points
at origin is
always misclassified



(c)

decision boundary slope
 $\frac{x_2}{x_1}$ between 0 and -1.
 $\Rightarrow \frac{\theta_2}{\theta_1} \in (0, 1)$
 $\vec{\theta}$ point toward upper right.
e.g. $\vec{\theta} = [1, 2]^T$
can be of any length.



(d)

not possible.

After drawing a decision boundary, how to find $\vec{\theta}$?

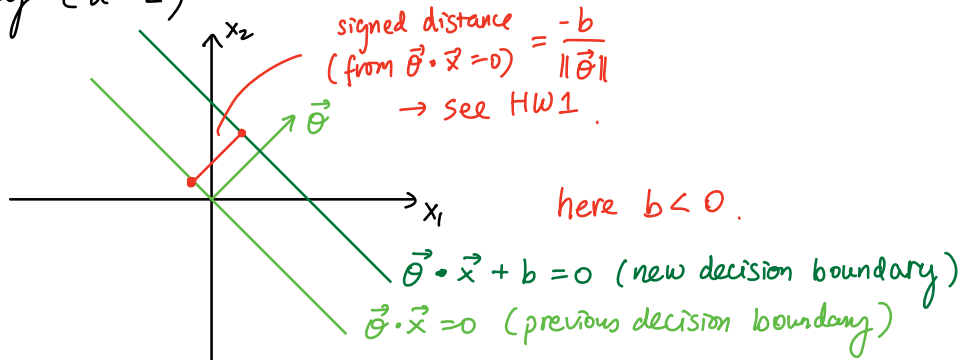
- method 1: eyeball the perpendicular direction.
- method 2: find a point on the decision boundary, it satisfies $\vec{\theta} \cdot \vec{x} = 0$.
solve for θ_1/θ_2 , determine the positive direction.

Linear Classifier with offset

$$h(\vec{x}; \vec{\theta}, b) = \text{sign}(\vec{\theta} \cdot \vec{x} + b)$$

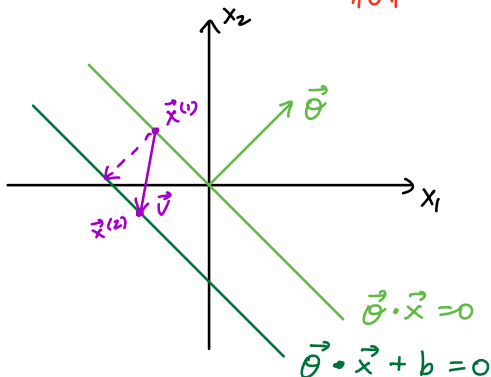
$$\vec{x} \in \mathbb{R}^d, \vec{\theta} \in \mathbb{R}^d, \underbrace{b \in \mathbb{R}}_{\text{offset aka intercept.}}$$

Graphically ($d=2$)



- The new decision boundary $\vec{\theta} \cdot \vec{x} + b = 0$ doesn't pass through the origin (when $b \neq 0$).
- $\vec{\theta} \cdot \vec{x} + b = 0$ is "parallel" to $\vec{\theta} \cdot \vec{x} = 0$.
 - they have the same normal vector $\vec{\theta}$
 - if they intersect, then there is some pt satisfying $\vec{\theta} \cdot \vec{x} + b = \vec{\theta} \cdot \vec{x}$ which is only possible when $b=0$

How do we derive $\frac{-b}{\|\vec{\theta}\|}$?



- pick a pt on old decision boundary
 $\vec{x}^{(1)}$ satisfies $\vec{\theta} \cdot \vec{x}^{(1)} = 0$
- pick a pt on new decision boundary.
 $\vec{x}^{(2)}$ satisfies $\vec{\theta} \cdot \vec{x}^{(2)} = -b$

- let $\vec{v} = \vec{x}^{(2)} - \vec{x}^{(1)}$
- now project $\vec{v}$ onto direction of $\vec{\theta}$

$$\text{proj}_{\vec{\theta}} \vec{v} = \left(\vec{v} \cdot \frac{\vec{\theta}}{\|\vec{\theta}\|} \right) \frac{\vec{\theta}}{\|\vec{\theta}\|}$$

$\frac{\vec{\theta}}{\|\vec{\theta}\|}$ is the unit vector in direction of $\vec{\theta}$.

- therefore, signed distance is simply

$$\begin{aligned} \vec{v} \cdot \frac{\vec{\theta}}{\|\vec{\theta}\|} &= \frac{(\vec{x}^{(2)} - \vec{x}^{(1)}) \cdot \vec{\theta}}{\|\vec{\theta}\|} = \frac{\vec{x}^{(2)} \cdot \vec{\theta} - \vec{x}^{(1)} \cdot \vec{\theta}}{\|\vec{\theta}\|} \\ &= \frac{-b - 0}{\|\vec{\theta}\|} = \frac{-b}{\|\vec{\theta}\|} \end{aligned}$$

Recap

- Linear classifier parameterized by $\vec{\theta}$:

$$h(\vec{x}; \vec{\theta}) = \text{sign}(\vec{\theta} \cdot \vec{x})$$

- training error given training data $D = \{\vec{x}^{(i)}, y^{(i)}\}_{i=1}^N$

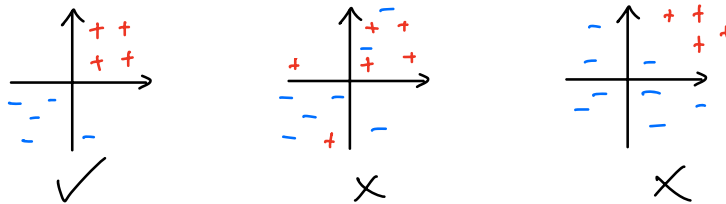
$$\mathcal{E}_N(\vec{\theta}) = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[y^{(i)}(\vec{\theta} \cdot \vec{x}^{(i)}) \leq 0]$$

Goal: find $\vec{\theta}^* = \underset{\vec{\theta}}{\text{argmin}} \mathcal{E}_N(\vec{\theta})$.

How?

- In general, this is not easy to solve (it's NP-hard)
- For now, consider a special case - linearly separable

Definition: training examples $D = \{\vec{x}^{(i)}, y^{(i)}\}_{i=1}^N$ are linearly separable through the origin if there exists parameter $\vec{\theta}$ such that $y^{(i)}(\vec{\theta} \cdot \vec{x}^{(i)}) > 0 \ \forall i = 1 \dots N$.



Perceptron Algorithm

Mistake-driven: starts with $\vec{\theta} = \vec{0}$ ← zero vector
tries to update $\vec{\theta}$ to correct any mistakes.

$$k=0, \vec{\theta}^{(0)} = \vec{0}$$

while not all pts. are correctly classified
for $i = 1 \dots N$

$$\text{if } y^{(i)}(\vec{\theta}^{(k)} \cdot \vec{x}^{(i)}) \leq 0 \leftarrow$$

$\vec{\theta}^{(k)}$ denotes
parameters
after k mistakes

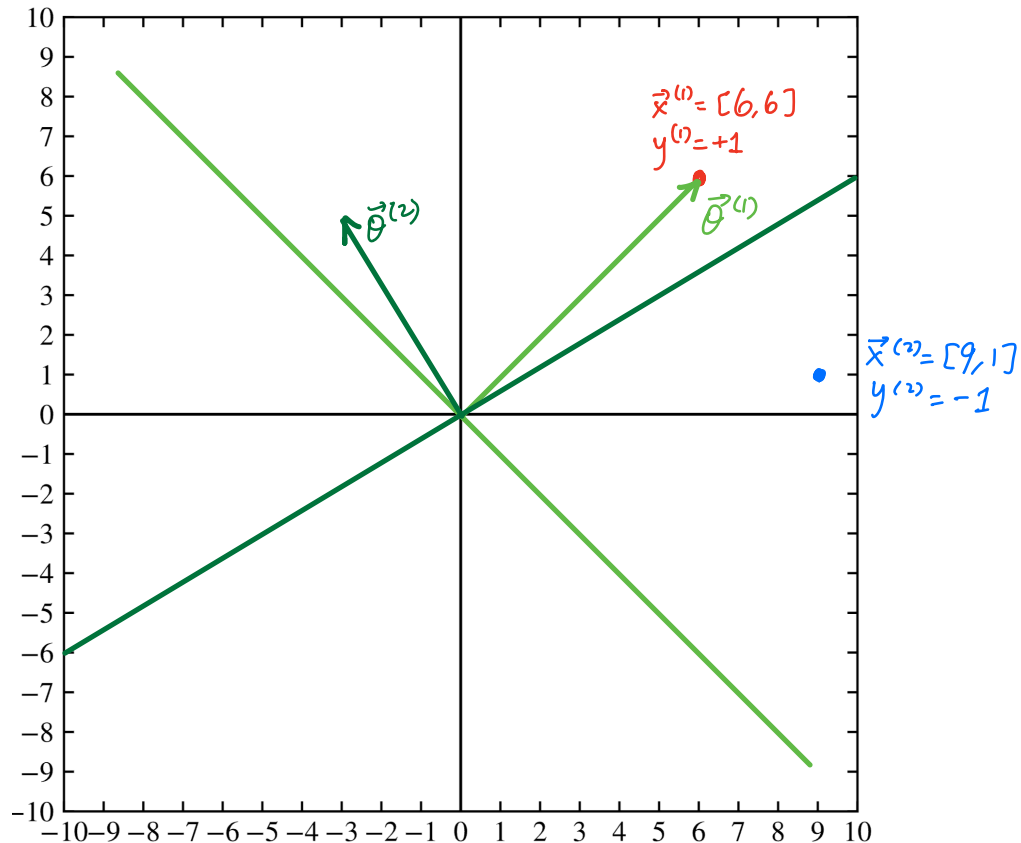
update rule

$$\vec{\theta}^{(k+1)} = \vec{\theta}^{(k)} + y^{(i)} \vec{x}^{(i)}$$

$$k++$$

parameters updated
only if we make a mistake

Worked Example



Initialization: $\vec{\theta}^{(0)} = [0, 0]^T$

First update: $\vec{\theta}^{(1)} = [0, 0] + [6, 6] = [6, 6]$

Second update: $\vec{\theta}^{(2)} = [6, 6] - [9, 1] = [-3, 5]$

Here, the classifier correctly classifies each pt after seeing them once.
and the algorithm converges after one (outer) loop.

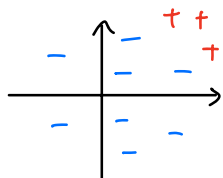
see notes
at the end

Theorem:

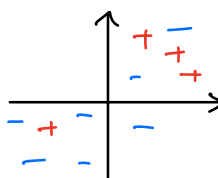
the perceptron algorithm converges after a finite number of mistakes
if the training examples are linearly separable (through the origin).

However, the solution may not be unique;
each point may be seen more than once or not at all.

What if not linearly separable through the origin?



Easy case: add an offset.



Hard case: next lecture.

Linear separability with offset: if there exist $\vec{\theta}$ and b such that
 $y^{(i)}(\vec{\theta} \cdot \vec{x}^{(i)} + b) > 0 \quad \forall i = 1 \dots N$

Perceptron with offset

$k=0, \vec{\theta}^{(0)} = \vec{0}, b^{(0)} = 0.$
 while not all pts correct (with $\vec{\theta}^{(k)}, b^{(k)}$)
 for $i = 1 \dots N$
 if $y^{(i)}(\vec{\theta}^{(k)} \cdot \vec{x}^{(i)} + b^{(k)}) \leq 0$
 $\vec{\theta}^{(k+1)} = \vec{\theta}^{(k)} + y^{(i)} \vec{x}^{(i)}$
 $b^{(k+1)} = b^{(k)} + y^{(i)}$
 $k = k+1$

How did we derive this?

augment $\vec{x}' = [1, \vec{x}]^T$

$\vec{\theta}' = [b, \vec{\theta}]^T$

$\vec{\theta}' \cdot \vec{x}' = b + \vec{\theta} \cdot \vec{x}$

and then apply the same update rule.

→ will implement in HW2.

Question: Is it possible for the algorithm to undershoot? Yes.

Suppose we make a mistake on $\vec{x}^{(i)}$ and update.

$$\vec{\theta}^{(k+1)} = \vec{\theta}^{(k)} + y^{(i)} \vec{x}^{(i)}$$

Consider classifying this exact same example after the update.

$$y^{(i)}(\vec{\theta}^{(k+1)} \cdot \vec{x}^{(i)}) \leq 0 \quad \text{since we undershoot.}$$

$$y^{(i)}((\vec{\theta}^{(k)} + y^{(i)} \vec{x}^{(i)}) \cdot \vec{x}^{(i)}) \leq 0$$

$$y^{(i)}(\vec{\theta}^{(k)} \cdot \vec{x}^{(i)}) + \underbrace{y^{(i)} y^{(i)}}_{=1} \underbrace{(\vec{x}^{(i)} \cdot \vec{x}^{(i)})}_{= \|\vec{x}^{(i)}\|^2} \leq 0$$

$$\underbrace{y^{(i)}(\vec{\theta}^{(k)} \cdot \vec{x}^{(i)})}_{\text{previous solution}} + \underbrace{\|\vec{x}^{(i)}\|^2}_{\text{increases by } \|\vec{x}^{(i)}\|^2 \text{ after each update}} \leq 0$$

so if $\|\vec{x}\|^2$ is small,
we may undershoot.

for example, $y^{(i)} = 1, \vec{x}^{(i)} = [-0.5, 0.5], \vec{\theta}^{(k)} = [1, 1]$.

Worked Example

